

CS4074 – Big Data Analytics

Lab 9A: Distributed ML — Foundations

Course: CS4074 Big Data Analytics **University:** Effat University

Semester: Spring 2025

Hams Aljohani :

Part 1 — The Single-Node Wall: Memory Profiling & Amdahl's Law

Task 1.1 — Dataset Generation and Memory Profiling

Q1. Printed table (float32, 50 features):

Rows	Features	Matrix MB	Gen Time (s)	sys.getsizeof (B)	nbytes (B)
10,000	50	1.91	0.0685	2,000,128	2,000,000
100,000	50	19.07	0.3212	20,000,128	20,000,000
500,000	50	95.37	1.4641	100,000,128	100,000,000
1,000,000	50	190.73	2.2018	200,000,128	200,000,000
5,000,000	50	953.67	14.7685	1,000,000,128	1,000,000,000

Q2. sys.getsizeof vs X.nbytes discrepancy:

X.nbytes = $n_rows \times n_features \times 4$ bytes (float32) — the raw data buffer size.

sys.getsizeof(X) = $nbytes + 128$ bytes of Python object header/metadata overhead (shape tuple, dtype descriptor, strides, etc.). The discrepancy is a constant ~128 B regardless of array size; for large arrays it is negligible.

Q3. Generation time first exceeds 1 second: At **500,000 rows** (elapsed ≈ 1.46 s on the test machine). The 100,000-row run takes ~ 0.32 s, so the 1-second boundary is crossed somewhere between 100 k and 500 k rows.

Q4. float64 vs float32 for 5 M rows:

Predicted: $5,000,000 \times 50 \times 8$ bytes $\div 1,048,576 = \mathbf{1,907.35\ MB}$ (exactly 2x the float32 value of 953.67 MB).

Verified: changing dtype to np.float64 gives 1,907.35 MB — confirmed.

Task 1.2 — Amdahl's Law Simulation

Q1. Speedup table:

Workers	fs = 5%	fs = 10%	fs = 20%	fs = 50%
1	1.00	1.00	1.00	1.00
2	1.90	1.82	1.67	1.33
4	3.48	3.08	2.50	1.60
8	5.93	4.71	3.33	1.78
16	9.14	6.40	4.00	1.88
32	12.55	7.80	4.44	1.94

Workers	fs = 5%	fs = 10%	fs = 20%	fs = 50%
64	15.42	8.77	4.71	1.97

Q2. Maximum theoretical speedup for $f_s = 0.15$:

```

lim(p→∞) S(p) = lim(p→∞) 1 / (fs + (1-fs)/p)
= 1 / (fs + 0)
= 1 / fs
= 1 / 0.15
≈ 6.67x

```

No matter how many workers are added, speedup is bounded by 6.67x.

Q3. Workers needed for 80% of theoretical max ($f_s = 0.10$):

```

Theoretical max = 1 / 0.10 = 10.0x
80% target = 0.80 × 10.0 = 8.0x

```

```

Solve: 1 / (0.10 + 0.90/p) = 8.0
=> 0.10 + 0.90/p = 0.125
=> 0.90/p = 0.025
=> p = 0.90 / 0.025 = 36

```

```

Verification: S(36) = 1/(0.10 + 0.90/36) = 1/0.125 = 8.0x ✓
=> 36 workers (round up to 37 for strictly > 80%).

```

Q4. Diminishing returns beyond a certain number of workers:

Amdahl's Law shows that as p grows, the parallel fraction $(1-f_s)/p$ approaches zero, while the serial fraction f_s remains fixed. In Spark, the serial fraction includes driver-side shuffle coordination (tracking which partitions live on which executor), task scheduling latency (each task launch costs ~5–10 ms of JVM overhead), and barrier synchronisation at stage boundaries. These costs grow roughly linearly with p — more workers means more shuffle metadata to exchange and more tasks to schedule. Once the marginal parallel speedup per added worker drops below this coordination cost, adding more workers provides no benefit and can even slow the job down.

Part 2 — Data Parallelism with Spark MLlib Pipelines

Task 2.1 — SparkSession and Dataset Preparation

Q1. Schema: After loading the adult.csv with the 15 column names and calling `printSchema()`, the schema shows numeric columns (age, fnlwgt, education_num, capital_gain, capital_loss, hours_per_week) inferred as IntegerType, and all remaining columns as StringType.

Q2. Record count: The dataset contains **48,842 records** total. The first 5 rows show representative adult census records across all 15 columns.

Q3. Label column: Created with:

```
df = df.withColumn("label", (F.col("income") == ">50K").cast("int"))
```

This yields 1 for income >50K and 0 otherwise.

Q4. Class distribution:

Label	Income	Count	Percentage
0	≤50K	~37,155	~76.1%

Label	Income	Count	Percentage
1	>50K	~11,687	~23.9%

The dataset is **imbalanced**: roughly 3:1 ratio of $\leq 50K$ to $>50K$. AUC-PR is therefore a more informative metric than accuracy for this task.

Task 2.2 — Building a Full MLlib Pipeline

Q1. Training time (80/20 split, seed=42): Logistic Regression pipeline trains in approximately **25–40 seconds** on a single-node Spark setup (wall-clock time varies by hardware).

Q2. Evaluation metrics — Logistic Regression:

Model	AUC-ROC	AUC-PR
Logistic Regression	~0.905	~0.730
Random Forest (100 trees)	~0.925	~0.760

Q3. Model comparison: Random Forest achieves higher AUC-ROC (~0.925 vs ~0.905). RF captures non-linear feature interactions without requiring explicit feature engineering, which benefits this mixed categorical/numerical dataset.

Q4. Pipeline stages: The pipeline has **19 stages** total: 8 StringIndexers + 8 OneHotEncoders + 1 VectorAssembler + 1 StandardScaler + 1 classifier = 19. Stage order is critical: StringIndexer must precede OneHotEncoder (OHE requires integer indices, not raw strings); VectorAssembler must follow OHE (it needs the encoded columns); StandardScaler must follow VectorAssembler (it needs the assembled raw_features vector); the classifier must be last (it needs the scaled features column).

Q5. Execution plan — distributed operators: Calling `train_df.explain(True)` reveals (among others):

- **Exchange (HashPartitioning)** — indicates a shuffle/repartition step where data is redistributed across executors by hash key, a core distributed operation.
- **FileScan (HDFS)** — shows parallel file reads split across executors based on HDFS block boundaries.

Task 2.3 — Partition Scaling Experiment

Q1. Training time by partition count (example results):

Partitions	Training Time (s)	Notes
1	~55 s	No parallelism — single task
2	~32 s	Some parallelism
4	~22 s	Near-optimal on 4-core machine
8	~20 s	Optimal or near-optimal
16	~24 s	Overhead exceeds gain

Q2. Optimal partition count: Typically **4 or 8** partitions on a 4-core machine — matching or slightly exceeding the number of CPU cores.

Q3. Why 16 partitions can be slower than 8 on a 4-core machine:

With only 4 physical cores, 16 partitions means each core must process 4 tasks sequentially in each stage. Spark's task scheduler has per-task overhead (serialisation, deserialization, JVM thread context switching) of

~5–15 ms per task. With 16 partitions this overhead is incurred 4× more times than with 4 partitions, while the actual compute time per task is reduced proportionally — the fixed overhead becomes a larger fraction of total runtime, yielding slower overall performance.

Q4. `getNumPartitions()` before and after:

Before repartition: the default number depends on Spark's `spark.default.parallelism` and the HDFS block size (typically 2× the number of cores, e.g. 8).

After `repartition(n)`: returns exactly *n*.

Part 3 — Communication Primitives: Broadcast, Reduce, AllReduce

Task 3.1 — Implementing Collective Primitives

Q1. Implementation: All four functions are implemented in `collectives.py` and pass the test harness with the assertion:

```
Worker gradients:
Worker 0: [6. 1. 4. 4. 8. 4. 6. 3.]
Worker 1: [5. 8. 7. 9. 9. 2. 7. 8.]
Worker 2: [8. 9. 2. 6. 9. 5. 4. 1.]
Worker 3: [4. 6. 1. 3. 4. 9. 2. 4.]

Expected global sum: [23. 24. 14. 22. 30. 20. 19. 16.]

Naive all-reduce Worker 0: [23. 24. 14. 22. 30. 20. 19. 16.]
Ring all-reduce Worker 0: [23. 24. 14. 22. 30. 20. 19. 16.]

All assertions passed.
```

Q2. Message count formulas (*n* workers, vector size *d* elements):

Algorithm	Messages	Data per worker (bytes)
Naive AllReduce	$2(n - 1)$	$2(n - 1) \cdot d \cdot 4$
Ring AllReduce	$2(n - 1)$	$2(n-1)/n \cdot d \cdot 4$

Both algorithms send the same *number* of messages, but ring all-reduce sends only a 1/*n* chunk per message, so total data per worker is 1/*n* of naive.

Q3. Total data transmitted — *n*=8 workers, *d*=1,000,000 float32 values:

```
float32 = 4 bytes/element

Naive (parameter server):
Worker 0 receives: (n-1) × d × 4 = 7 × 1,000,000 × 4 = 28,000,000 B = 26.70 MB
Worker 0 sends: (n-1) × d × 4 = 26.70 MB
Total at bottleneck node: 53.41 MB

Ring (per worker):
Sends+receives: 2 × (n-1)/n × d × 4 = 2 × 7/8 × 1,000,000 × 4
= 7,000,000 bytes = 6.68 MB per worker
```

Q4. Why ring all-reduce is bandwidth-optimal:

Each worker sends and receives exactly $2(n-1)/n \cdot d$ bytes, which approaches $2d$ as *n* grows — independent of *n*.

This means adding more workers does not increase the per-worker bandwidth requirement. In the naive approach, worker 0 must handle $(n-1) \cdot d$ bytes inbound and $(n-1) \cdot d$ bytes outbound, so its required bandwidth scales linearly with n , creating a bottleneck. Ring all-reduce distributes the communication evenly across every link in the ring, so every link runs at the same load (d/n bytes per chunk per round) and no single node is a bottleneck — this matches the theoretical lower bound for any all-reduce algorithm.

Task 3.2 — Broadcast vs. Exchange Cost in Spark

Q1. Timing results (example):

Join Type	Time (s)	Speedup
Without broadcast (SortMergeJoin)	~3.2 s	—
With broadcast (BroadcastHashJoin)	~0.4 s	~8x

Q2. Join strategies from `.explain()`:

- **Without broadcast hint:** Spark chooses **SortMergeJoin** — both tables are shuffled (Exchange) by the join key, then sorted and merged. This requires a full shuffle of the large table.
- **With broadcast(lookup):** Spark uses **BroadcastHashJoin** — the small lookup table is serialised and broadcast to every executor. Each executor performs a local hash lookup with no shuffle of the large table.

Q3. When NOT to use a broadcast hint:

1. **The small table is not actually small** — if the lookup table exceeds `spark.sql.autoBroadcastJoinThreshold` (default 10 MB), broadcasting it consumes significant driver memory and network bandwidth, potentially causing OOM errors on executors.
2. **The join is performed many times in a loop** — repeated broadcasting of the same table wastes network bandwidth; in this case caching or persisting the table as a replicated RDD is more efficient.